

Code Complexity Analysis - Nearest Neighbors

Index

1	Pseudocode	2
1.1	Function 1: nearestNeighbors	2
1.2	Function 2: search	2
1.3	Function 3: searchSubtrees	3
1.4	Function 4: getCoordinate	3
1.5	Function 5: nearestNeighbors	3
1.6	Function 6: nearestNeighbor	3
1.7	Function 7: nearestNeighbor	4
2	Time Complexity	5
2.1	Function: nearestNeighbors	5
2.2	Function: search	5
2.3	Function: searchSubtrees	5
2.4	Function: getCoordinate	6
2.5	Function: nearestNeighbors(lat, lon, n)	6
2.6	Function: nearestNeighbor(lat, lon, filters...)	6
2.7	Function: nearestNeighbor(lat, lon)	7

1 Pseudocode

1.1 Function 1: nearestNeighbors

Location: KD-Tree Implementation

```
1 FUNCTION nearestNeighbors(lat, lon, n, filters...):
2   IF root IS NULL OR n <= 0:
3     RETURN empty list
4
5   filterSet = SET(filters) IF filters NOT EMPTY ELSE empty set
6
7   candidates = MAX-HEAP (size n) ordered by distance DESC
8
9   search(root, lat, lon, n, 0, candidates, filterSet)
10
11  result = empty list
12
13  WHILE candidates NOT EMPTY:
14    INSERT candidates.poll().nodeData AT FRONT OF result
15
16  RETURN result
17 END FUNCTION
```

1.2 Function 2: search

Location: KD-Tree Implementation

```
1 FUNCTION search(node, targetLat, targetLon, n, depth, candidates, filterSet):
2   IF node IS NULL:
3     RETURN
4
5   nodeData = node.element
6
7   IF filterSet NOT EMPTY:
8     nodeTimeZone = nodeData.stations.first.timeZoneGroup
9     IF nodeTimeZone IS NULL OR nodeTimeZone NOT IN filterSet:
10      searchSubtrees(node, targetLat, targetLon, n, depth, candidates,
11      filterSet)
12      RETURN
13
14   nodeLat = nodeData.coordinate.latitude
15   nodeLon = nodeData.coordinate.longitude
16
17   dist = haversineDistance(nodeLat, nodeLon, targetLat, targetLon)
18
19   IF candidates.size < n:
20     candidates.add(nodeData, dist)
21   ELSE IF dist < candidates.peek().distance:
22     candidates.poll()
23     candidates.add(nodeData, dist)
24
25   searchSubtrees(node, targetLat, targetLon, n, depth, candidates, filterSet)
26 END FUNCTION
```

1.3 Function 3: searchSubtrees

Location: KD-Tree Implementation

```
1 FUNCTION searchSubtrees(node, targetLat, targetLon, n, depth, candidates,
2   filterSet):
3   compareLatitude = (depth MOD 2 == 0)
4
5   nodeSplitValue = (latitude OR longitude BASED ON compareLatitude)
6   targetSplitValue = targetLat IF compareLatitude ELSE targetLon
7
8   targetIsLeft = (targetSplitValue < nodeSplitValue)
9
10  nearSubtree = node.left IF targetIsLeft ELSE node.right
11  farSubtree = node.right IF targetIsLeft ELSE node.left
12
13  search(nearSubtree, targetLat, targetLon, n, depth + 1, candidates,
14  filterSet)
15
16  IF candidates NOT EMPTY:
17    distanceToPlane = targetSplitValue - nodeSplitValue
18    IF (distanceToPlane^2) < candidates.peek().distance:
19      search(farSubtree, targetLat, targetLon, n, depth + 1, candidates,
20      filterSet)
21  END FUNCTION
```

1.4 Function 4: getCoordinate

Location: KD-Tree Implementation

```
1 FUNCTION getCoordinate(node, getLatitude):
2   nodeData = node.element
3   RETURN nodeData.coordinate.latitude IF getLatitude
4   ELSE nodeData.coordinate.longitude
5 END FUNCTION
```

1.5 Function 5: nearestNeighbors

Location: KD-Tree Implementation

```
1 FUNCTION nearestNeighbors(lat, lon, n):
2   RETURN nearestNeighbors(lat, lon, n, empty TimeZoneGroup array)
3 END FUNCTION
```

1.6 Function 6: nearestNeighbor

Location: KD-Tree Implementation

```
1 FUNCTION nearestNeighbor(lat, lon, filters...):
2   neighbors = nearestNeighbors(lat, lon, 1, filters)
3
4   IF neighbors IS EMPTY:
5     RETURN null
6
7   RETURN neighbors[0]
8 END FUNCTION
```

1.7 Function 7: nearestNeighbor

Location: KD-Tree Implementation

```
1 FUNCTION nearestNeighbor(lat, lon):  
2     RETURN nearestNeighbor(lat, lon, empty TimeZoneGroup array)  
3 END FUNCTION
```

2 Time Complexity

2.1 Function: nearestNeighbors

Operation	Description	Best Case	Average Case	Worst Case	Notes
filterSet = SET(filters)	Create set from filters	O(1)	O(1)	O(1)	Filters constant-size
candidates = MAX-HEAP	Initialize heap	O(1)	O(1)	O(1)	Heap grows to size n
search(root, ...)	KD-tree recursive search	O(log n)	O(log n)	O(n)	Depends on pruning
WHILE candidates NOT EMPTY	Build final result list	O(n)	O(n)	O(n)	n = requested neighbors
TOTAL		O(log n)	O(log n)	O(n)	KD-tree search dominates

2.2 Function: search

Operation	Description	Best Case	Average Case	Worst Case	Notes
IF node IS NULL	Null check, stop recursion	O(1)	O(1)	O(1)	-
IF filterSet NOT EMPTY	Time zone filter check	O(1)	O(1)	O(1)	Constant-time
dist = haversineDistance(	Compute distance	O(1)	O(1)	O(1)	Always computed
candidates.add / poll / peek	Update heap	O(1)	O(1)	O(1)	n is constant
searchSubtrees(...)	Continue recursion	O(log n)	O(log n)	O(n)	KD-tree traversal
TOTAL		O(log n)	O(log n)	O(n)	Recursive traversal dominates

2.3 Function: searchSubtrees

Operation	Description	Best Case	Average Case	Worst Case	Notes
compareLatitude = (depth MOD 2...)	Select axis	O(1)	O(1)	O(1)	-
targetIsLeft = (...)	Compare split values	O(1)	O(1)	O(1)	-
search(nearSubtree ...)	Always explore nearer subtree	O(1)	O(log n)	O(n)	-
distanceToPlane = ...	Compute plane distance	O(1)	O(1)	O(1)	-

search(farSubtree, ...)	Explore far subtree if needed	$O(0)$	$O(\log n)$	$O(n)$	No pruning → worst case
TOTAL		$O(1)$	$O(\log n)$	$O(n)$	Pruning effectiveness varies

2.4 Function: getCoordinate

Operation	Description	Best Case	Average Case	Worst Case	Notes
RETURN coordinate.latitude OR longitude	Access field	$O(1)$	$O(1)$	$O(1)$	Simple lookup
TOTAL		$O(1)$	$O(1)$	$O(1)$	Constant time operation

2.5 Function: nearestNeighbors(lat, lon, n)

Operation	Description	Best Case	Average Case	Worst Case	Notes
RETURN nearestNeighbors(lat, lon, n, empty...)	Wrapper call	$O(1)$	$O(1)$	$O(1)$	Delegates work
TOTAL		$O(1)$	$O(1)$	$O(1)$	Delegate to main function

2.6 Function: nearestNeighbor(lat, lon, filters...)

Operation	Description	Best Case	Average Case	Worst Case	Notes
neighbors = nearestNeighbors(lat, lon, filters..., k = 1 ...)	Delegate work	$O(1)$	$O(1)$	$O(1)$	Single nearest node
RETURN neighbors[0]	Constant return	$O(1)$	$O(1)$	$O(1)$	-
TOTAL		$O(1)$	$O(1)$	$O(1)$	Wrapper function

2.7 Function: nearestNeighbor(lat, lon)

Operation	Description	Best Case	Average Case	Worst Case	Notes
RETURN nearestNeighbor(lat, lon, empty filters)	Wrapper call	$O(1)$	$O(1)$	$O(1)$	-
TOTAL		$O(1)$	$O(1)$	$O(1)$	Final wrapper